



COURS COMPLET PYTHON - PRÉPARATION EXAMEN ÉCRIT



TABLE DES MATIÈRES

1. [Le Cours - Notions Fondamentales](#)
 2. [Règles de Syntaxe Essentielles](#)
 3. [Exercices Progressifs](#)
 4. [Pièges Courants d'Examen](#)
-



PARTIE 1 : LE COURS COMPLET

1.1 VARIABLES ET TYPES DE DONNÉES

Concept

Une **variable** est un conteneur qui stocke une valeur. C'est comme une boîte étiquetée où on met des données.

Syntaxe exacte à retenir

```
nom_variable = valeur
```

Les types de données en Python

Les nombres

```
# Entier (int)
age = 25
nombre = -10
zero = 0

# Nombre flottant (float) - avec virgule/point décimal
prix = 19.99
temperature = -5.5
pi = 3.14159
```

Les chaînes de caractères (str)

```
# Avec guillemets simples
nom = 'Alice'
ville = 'Paris'

# Avec guillemets doubles
prenom = "Bob"
message = "Bonjour à tous"

# Avec triple guillemets (texte multi-ligne)
texte = """Ceci est un texte
sur plusieurs lignes
en Python"""
```

Les booléens (bool)

```
# Deux seules valeurs possibles
vrai = True
faux = False

# Exemple concret
est_majeur = True
est_connecte = False
```

Afficher des valeurs : print()

```
# Afficher une valeur
print(25)

# Afficher un texte
print("Bonjour")

# Afficher plusieurs valeurs (séparées par des espaces)
print("J'ai", 25, "ans")

# Afficher le contenu d'une variable
nom = "Alice"
print(nom)
```

Opérations sur les nombres

```
# Addition
somme = 10 + 5 # résultat : 15

# Soustraction
difference = 20 - 8 # résultat : 12
```

```
# Multiplication
produit = 4 * 7 # résultat : 28

# Division (toujours un float)
division = 15 / 3 # résultat : 5.0

# Division entière (arrondi vers le bas)
division_entiere = 17 // 5 # résultat : 3

# Modulo (le reste de la division)
reste = 17 % 5 # résultat : 2

# Puissance
puissance = 2 ** 3 # résultat : 8
```

Comparaisons (retournent True ou False)

```
# Égal
5 == 5 # True
5 == 3 # False

# Différent
5 != 3 # True
5 != 5 # False

# Strictement inférieur/supérieur
5 < 10 # True
5 > 10 # False

# Inférieur ou égal / Supérieur ou égal
5 <= 5 # True
5 >= 10 # False
```

1.2 CONDITIONS : IF / ELIF / ELSE

Concept

Les conditions permettent d'exécuter du code **seulement si** une condition est vraie.

Structure de base

```
if condition:
    # Code à exécuter si condition est True
    print("La condition est vraie")
```

Avec IF et ELSE

```
age = 25

if age >= 18:
    print("Vous êtes majeur")
else:
    print("Vous êtes mineur")
```

Avec IF, ELIF et ELSE

```
note = 15

if note >= 18:
    print("Excellent")
elif note >= 15:
    print("Très bien")
elif note >= 12:
    print("Bien")
else:
    print("À améliorer")
```

Conditions composées avec AND/OR/NOT

```
age = 25
permis = True

# AND - Les DEUX conditions doivent être vraies
if age >= 18 and permis == True:
    print("Vous pouvez conduire")

# OR - AU MOINS UNE condition doit être vraie
if age < 18 or permis == False:
    print("Vous ne pouvez pas conduire")

# NOT - Inverse la condition
if not permis == False:
    print("Vous avez le permis")
```

Imbrication de conditions

```
age = 25
ville = "Paris"

if age >= 18:
```

```
if ville == "Paris":
    print("Vous êtes majeur et à Paris")
else:
    print("Vous êtes majeur mais pas à Paris")
else:
    print("Vous êtes mineur")
```

1.3 BOUCLES : FOR ET WHILE

La boucle FOR

Concept

La boucle `for` répète un bloc de code **un nombre de fois défini**.

Avec range()

```
# Afficher de 0 à 4
for i in range(5):
    print(i)

# Afficher de 1 à 5
for i in range(1, 6):
    print(i)

# Afficher de 0 à 9 en avançant de 2
for i in range(0, 10, 2):
    print(i) # Affiche: 0, 2, 4, 6, 8
```

Parcourir une liste (voir section listes)

```
fruits = ["pomme", "banane", "orange"]
for fruit in fruits:
    print(fruit)
```

Boucle FOR avec break

```
for i in range(10):
    if i == 5:
        break # Arrête la boucle
    print(i) # Affiche: 0, 1, 2, 3, 4
```

Boucle FOR avec continue

```
for i in range(5):  
    if i == 2:  
        continue # Saute cette itération  
    print(i) # Affiche: 0, 1, 3, 4
```

La boucle WHILE

Concept

La boucle **while** répète un bloc de code **tant qu'une condition est vraie**.

Syntaxe de base

```
compteur = 0  
while compteur < 5:  
    print(compteur)  
    compteur = compteur + 1  
# Affiche: 0, 1, 2, 3, 4
```

Avec break

```
compteur = 0  
while True:  
    print(compteur)  
    compteur = compteur + 1  
    if compteur == 5:  
        break # Arrête la boucle
```

Avec continue

```
compteur = 0  
while compteur < 10:  
    compteur = compteur + 1  
    if compteur % 2 == 0: # Si pair  
        continue # Saute  
    print(compteur) # Affiche: 1, 3, 5, 7, 9
```

1.4 FONCTIONS

Concept

Une **fonction** est un bloc de code réutilisable qui effectue une tâche spécifique.

Définir une fonction sans paramètre

```
def dire_bonjour():  
    print("Bonjour à tous!")  
  
# Pour exécuter la fonction  
dire_bonjour()
```

Fonction avec paramètres

```
def dire_bonjour(nom):  
    print("Bonjour", nom)  
  
# Exécuter la fonction  
dire_bonjour("Alice") # Affiche: Bonjour Alice
```

Fonction avec plusieurs paramètres

```
def addition(a, b):  
    resultat = a + b  
    print("La somme est:", resultat)  
  
addition(5, 3) # Affiche: La somme est: 8
```

Fonction qui retourne une valeur (return)

```
def multiplier(a, b):  
    resultat = a * b  
    return resultat  
  
# Récupérer le résultat  
produit = multiplier(4, 5)  
print(produit) # Affiche: 20
```

Fonction avec plusieurs retours

```
def obtenir_max_min(a, b):  
    return a + b, a - b  
  
somme, difference = obtenir_max_min(10, 3)  
print(somme) # Affiche: 13  
print(difference) # Affiche: 7
```

Fonction avec valeur par défaut

```
def saluer(nom="Ami"):
    print("Bonjour", nom)

saluer()          # Affiche: Bonjour Ami
saluer("Sophie")  # Affiche: Bonjour Sophie
```

Portée des variables

```
x = 10 # Variable globale

def fonction():
    y = 5 # Variable locale (existe seulement dans la fonction)
    print(x) # Peut accéder à x
    print(y)

fonction()
# print(y) # ERREUR! y n'existe pas en dehors de la fonction
```

1.5 LISTES

Concept

Une **liste** est un ensemble ordonné de valeurs. On peut la modifier.

Créer une liste

```
# Liste vide
liste_vide = []

# Liste avec des nombres
nombres = [1, 2, 3, 4, 5]

# Liste avec des textes
fruits = ["pomme", "banane", "orange"]

# Liste mixte (plusieurs types)
mixte = [1, "texte", 3.14, True]

# Liste avec des listes
matrice = [[1, 2], [3, 4], [5, 6]]
```

Accéder aux éléments (indexation)

```
fruits = ["pomme", "banane", "orange"]

# L'indice commence à 0
print(fruits[0]) # Affiche: pomme
print(fruits[1]) # Affiche: banane
print(fruits[2]) # Affiche: orange

# Indice négatif (de la fin)
print(fruits[-1]) # Affiche: orange (dernier élément)
print(fruits[-2]) # Affiche: banane (avant-dernier)
```

Modifier un élément

```
fruits = ["pomme", "banane", "orange"]
fruits[0] = "kiwi"
print(fruits) # Affiche: ['kiwi', 'banane', 'orange']
```

Ajouter un élément

```
fruits = ["pomme", "banane"]
fruits.append("orange") # Ajoute à la fin
print(fruits) # Affiche: ['pomme', 'banane', 'orange']
```

Supprimer un élément

```
fruits = ["pomme", "banane", "orange"]
del fruits[1] # Supprime l'élément à l'indice 1
print(fruits) # Affiche: ['pomme', 'orange']
```

Longueur d'une liste

```
fruits = ["pomme", "banane", "orange"]
nombre = len(fruits)
print(nombre) # Affiche: 3
```

Parcourir une liste

```
fruits = ["pomme", "banane", "orange"]
```

```
for fruit in fruits:
    print(fruit)
```

Slicing (extraire une partie de liste)

```
nombres = [0, 1, 2, 3, 4, 5]

# Du début jusqu'à l'indice 3 (exclu)
print(nombres[0:3]) # Affiche: [0, 1, 2]

# De l'indice 2 à la fin
print(nombres[2:]) # Affiche: [2, 3, 4, 5]

# Les 3 premiers
print(nombres[:3]) # Affiche: [0, 1, 2]

# Tous les 2 éléments
print(nombres[::2]) # Affiche: [0, 2, 4]
```

Vérifier si un élément est dans la liste

```
fruits = ["pomme", "banane", "orange"]

if "pomme" in fruits:
    print("La pomme est dans la liste")

if "raisin" not in fruits:
    print("Le raisin n'est pas dans la liste")
```

1.6 DICTIONNAIRES

Concept

Un **dictionnaire** est une collection de **clé: valeur**. Chaque clé est unique.

Créer un dictionnaire

```
# Dictionnaire vide
dictionnaire_vide = {}

# Dictionnaire avec des données
personne = {
    "nom": "Alice",
    "age": 25,
    "ville": "Paris"
```

```
}

# Dictionnaire mixte
info = {
    "nom": "Bob",
    "age": 30,
    "scores": [8, 9, 7]
}
```

Accéder à une valeur

```
personne = {
    "nom": "Alice",
    "age": 25,
    "ville": "Paris"
}

print(personne["nom"])    # Affiche: Alice
print(personne["age"])    # Affiche: 25
print(personne["ville"])  # Affiche: Paris
```

Modifier une valeur

```
personne = {
    "nom": "Alice",
    "age": 25
}

personne["age"] = 26
print(personne)  # Affiche: {'nom': 'Alice', 'age': 26}
```

Ajouter une nouvelle clé-valeur

```
personne = {"nom": "Alice", "age": 25}
personne["ville"] = "Lyon"
print(personne)
# Affiche: {'nom': 'Alice', 'age': 25, 'ville': 'Lyon'}
```

Supprimer une clé-valeur

```
personne = {"nom": "Alice", "age": 25, "ville": "Paris"}
del personne["ville"]
print(personne)  # Affiche: {'nom': 'Alice', 'age': 25}
```

Vérifier si une clé existe

```
personne = {"nom": "Alice", "age": 25}

if "nom" in personne:
    print("La clé 'nom' existe")

if "email" not in personne:
    print("La clé 'email' n'existe pas")
```

Parcourir un dictionnaire

```
personne = {"nom": "Alice", "age": 25, "ville": "Paris"}

# Parcourir les clés
for cle in personne:
    print(cle) # Affiche: nom, age, ville

# Parcourir les clés et valeurs
for cle, valeur in personne.items():
    print(cle, ":", valeur)
# Affiche: nom : Alice
#           age : 25
#           ville : Paris
```

Méthodes utiles sur les dictionnaires

```
personne = {"nom": "Alice", "age": 25}

# Obtenir toutes les clés
cles = personne.keys() # dict_keys(['nom', 'age'])

# Obtenir toutes les valeurs
valeurs = personne.values() # dict_values(['Alice', 25])

# Obtenir les paires clé-valeur
paires = personne.items() # dict_items([('nom', 'Alice'), ('age', 25)])
```

1.7 GESTION DES ERREURS : TRY / EXCEPT

Concept

La gestion des erreurs permet d'attraper les problèmes et de continuer le programme au lieu qu'il s'arrête.

Structure de base

```
try:
    # Code qui pourrait causer une erreur
    resultat = 10 / 2
    print(resultat)
except:
    # Code exécuté s'il y a une erreur
    print("Une erreur s'est produite!")
```

Attraper un type d'erreur spécifique

```
try:
    nombre = int("abc") # Erreur! "abc" n'est pas un nombre
except ValueError:
    print("Erreur: Vous devez entrer un nombre valide")
except ZeroDivisionError:
    print("Erreur: Division par zéro impossible")
```

Types d'erreurs courants

```
# ValueError - Valeur invalide
try:
    age = int("vingt") # ERREUR
except ValueError:
    print("Pas un nombre!")

# ZeroDivisionError - Division par zéro
try:
    resultat = 10 / 0 # ERREUR
except ZeroDivisionError:
    print("Impossible de diviser par zéro!")

# IndexError - Indice hors limites
try:
    liste = [1, 2, 3]
    print(liste[10]) # ERREUR
except IndexError:
    print("Indice inexistant!")

# KeyError - Clé inexistante
try:
    dico = {"nom": "Alice"}
    print(dico["age"]) # ERREUR
except KeyError:
    print("Clé inexistante!")

# TypeError - Type incorrect
try:
    resultat = "texte" + 5 # ERREUR
```

```
except TypeError:
    print("Types incompatibles!")
```

Utiliser ELSE et FINALLY

```
try:
    nombre = int("123")
    resultat = 100 / nombre
except ValueError:
    print("Erreur: Pas un nombre")
except ZeroDivisionError:
    print("Erreur: Division par zéro")
else:
    print("Succès! Résultat:", resultat) # Si pas d'erreur
finally:
    print("Fin du bloc try/except") # Toujours exécuté
```

Exemple complet

```
while True:
    try:
        age = int(input("Entrez votre âge: "))
        if age < 0:
            print("L'âge ne peut pas être négatif!")
            continue
        print(f"Vous avez {age} ans")
        break
    except ValueError:
        print("Veuillez entrer un nombre!")
```



PARTIE 2 : RÈGLES DE SYNTAXE ESSENTIELLES POUR L'EXAMEN

À ABSOLUMENT RETENIR

✓ L'INDENTATION

- **L'indentation est OBLIGATOIRE** en Python (4 espaces ou 1 tabulation)
- Chaque bloc (if, for, while, def, try, etc.) doit être indenté
- ✗ ERREUR COMMUNE: Oublier l'indentation

```
# BON
if age >= 18:
    print("Majeur") # Indenté
else:
    print("Mineur") # Indenté

# MAUVAIS
if age >= 18:
print("Majeur") # PAS indenté = ERREUR!
```

✓ LES DEUX-POINTS 😊

- Chaque structure de contrôle (if, for, while, def, try) **doit se terminer par :**
- ✗ ERREUR COMMUNE: Oublier le deux-points

```
# BON
if age >= 18:
    print("OK")

for i in range(5):
    print(i)

def fonction():
    pass

# MAUVAIS
if age >= 18 # MANQUE le :
    print("OK")
```

✓ LES NOMS DE VARIABLES

- Commencent par une **lettre ou underscore** (`_`)
- Contiennent **uniquement**: lettres, chiffres, underscores
- **Sensible à la casse**: `nom` ≠ `Nom` ≠ `NOM`
- Ne peuvent pas être des mots-clés (if, for, while, etc.)

```
# BON
mon_age = 25
age2 = 30
_prive = "secret"

# MAUVAIS
2age = 30 # Commence par un chiffre
mon-age = 25 # Contient un tiret
if = 10 # mot-clé réservé
```

✓ LES PARENTHÈSES ET LES PARENTHÈSES CARRÉES

- Listes: `[ ]`
- Tuples: `( )`
- Appels de fonctions: `function()`
- Conditions: `if (condition):`

```
# BON
liste = [1, 2, 3]
tuple_data = (1, 2, 3)
print(liste)
if (age > 18):
    pass

# MAUVAIS
liste = (1, 2, 3) # Utilisé des parenthèses au lieu de []
```

✓ LES GUILLEMETS

- Utilisez des **guillemets simples** `'` ou **doubles** `"`
- **Soyez cohérent** dans le même code
- Pour du texte multiligne: **triple guillemets** `"""`

```
# BON
nom = "Alice"
prenom = 'Bob'
texte = """Ligne 1
Ligne 2"""

# MAUVAIS
nom = "Alice' # Guillemets non fermés
prenom = 'Bob" # Types différents
```

✓ LES ESPACES

- Pas de contrainte stricte, mais **8 caractères** avant et après les opérateurs est recommandé pour la lisibilité
- **Après les virgules**: mettez un espace

```
# BON
a = 5 + 3
liste = [1, 2, 3]
nom = "Alice"

# Acceptable mais moins lisible
```

```
a=5+3
liste=[1,2,3]
```

✓ RETURN vs PRINT

- `return`: Retourne une valeur depuis une fonction
- `print()`: Affiche une valeur à l'écran
- Ce ne sont PAS la même chose!

```
# RETOURNER une valeur
def calculer(a, b):
    return a + b # Retourne le résultat

resultat = calculer(5, 3) # resultat = 8

# AFFICHER une valeur
def afficher(a, b):
    print(a + b) # Affiche le résultat

afficher(5, 3) # Affiche 8, mais ne retourne rien
```

✓ COMPARAISON vs ATTRIBUTION

- Attribution: `=` (une seule égal)
- Comparaison: `==` (deux égaux)
- ✗ ERREUR TRÈS COMMUNE: Confondre les deux

```
# BON - Attribution
age = 25

# BON - Comparaison
if age == 25:
    print("OK")

# MAUVAIS - Attribution dans une comparaison
if age = 25: # ERREUR! Syntaxe incorrecte
    print("OK")
```

✓ LONGUEUR vs INDICE

- `len(liste)`: Donne la **longueur** (nombre d'éléments)
- `liste[indice]`: Accède à un élément
- Les indices commencent à **0**

```
fruits = ["pomme", "banane", "orange"]
```

```
len(fruits)      # 3 (il y a 3 éléments)
fruits[0]        # "pomme" (premier élément, indice 0)
fruits[1]        # "banane" (indice 1)
fruits[2]        # "orange" (indice 2)
fruits[3]        # ERREUR! Indice 3 n'existe pas
```



PARTIE 3 : EXERCICES PROGRESSIFS

NIVEAU 1 - DÉBUTANT

Exercice 1.1 : Variables et Affichage

Énoncé: Écrivez un programme qui:

1. Crée une variable contenant votre nom
2. Crée une variable contenant votre âge
3. Affiche: "Je m'appelle [nom] et j'ai [âge] ans"

Correction détaillée:

```
# Créer la variable pour le nom
nom = "Alice"

# Créer la variable pour l'âge
age = 25

# Afficher le résultat
print("Je m'appelle", nom, "et j'ai", age, "ans")

# Alternative avec formatage (plus avancé)
print(f"Je m'appelle {nom} et j'ai {age} ans")
```

Ce qu'il faut comprendre:

- On crée des variables avec `=`
- `print()` affiche plusieurs valeurs séparées par des virgules
- Le f-string `f"..."` permet d'insérer des variables avec `{}`

Exercice 1.2 : Opérations Arithmétiques

Énoncé: Une pizza coûte 12€. Vous en achetez 3, et vous payez avec un billet de 50€. Écrivez un programme qui:

1. Calcule le prix total
2. Calcule la monnaie reçue

3. Affiche les résultats

Correction détaillée:

```
# Prix unitaire
prix_unitaire = 12

# Nombre de pizzas
nombre_pizzas = 3

# Somme payée
somme_payee = 50

# Calculer le prix total
prix_total = prix_unitaire * nombre_pizzas

# Calculer la monnaie
monnaie = somme_payee - prix_total

# Afficher les résultats
print("Prix total:", prix_total, "€")
print("Monnaie:", monnaie, "€")
```

Ce qu'il faut comprendre:

- La multiplication `*` pour calculer le total
- La soustraction `-` pour la monnaie
- `print()` peut afficher du texte et des nombres

Exercice 1.3 : Condition Simple

Énoncé: Écrivez un programme qui:

1. Demande l'âge de l'utilisateur
2. Dit si la personne est majeure (≥ 18) ou mineure

Correction détaillée:

```
# Demander l'âge
age = int(input("Quel est votre âge? "))

# Vérifier si majeur ou mineur
if age >= 18:
    print("Vous êtes majeur")
else:
    print("Vous êtes mineur")
```

Ce qu'il faut comprendre:

- `input()` demande quelque chose à l'utilisateur
 - `int()` convertit le texte en nombre
 - `if/else` exécute du code selon une condition
 - `>=` signifie "supérieur ou égal"
-

NIVEAU 2 - INTERMÉDIAIRE

Exercice 2.1 : Boucles et Listes

Énoncé: Écrivez un programme qui:

1. Crée une liste de 5 nombres
2. Affiche chaque nombre multiplié par 2
3. Affiche la somme totale de tous les nombres

Correction détaillée:

```
# Créer la liste de nombres
nombres = [2, 4, 6, 8, 10]

# Afficher chaque nombre multiplié par 2
print("Nombres multipliés par 2:")
for nombre in nombres:
    print(nombre * 2)

# Calculer la somme
somme = 0
for nombre in nombres:
    somme = somme + nombre

# Afficher la somme
print("Somme totale:", somme)
```

Alternative plus avancée:

```
nombres = [2, 4, 6, 8, 10]

# Utiliser sum() pour la somme
print("Somme totale:", sum(nombres))
```

Ce qu'il faut comprendre:

- `for nombre in liste:` parcourt chaque élément
 - Accumuler une somme avec `somme = somme + nombre`
 - `sum()` est une fonction intégrée pour calculer la somme
-

Exercice 2.2 : Dictionnaire et Conditions

Énoncé: Vous avez un dictionnaire avec les prix d'articles. Écrivez un programme qui:

1. Affiche les articles avec leur prix
2. Applique une remise de 10% si le prix > 100€
3. Affiche le prix après remise

Correction détaillée:

```
# Dictionnaire des articles et prix
articles = {
    "laptop": 1200,
    "souris": 25,
    "clavier": 80,
    "moniteur": 350
}

# Parcourir le dictionnaire
for article, prix in articles.items():
    print(f"Article: {article}, Prix: {prix}€")

    # Appliquer la remise si prix > 100
    if prix > 100:
        remise = prix * 0.10
        prix_final = prix - remise
        print(f"  Remise de 10%: {remise}€")
        print(f"  Prix final: {prix_final}€")
    else:
        print(f"  Prix final: {prix}€")

    print() # Ligne vide pour la lisibilité
```

Ce qu'il faut comprendre:

- `article, valeur in dictionnaire.items()` pour parcourir les paires clé-valeur
- Calculer le pourcentage: `prix * 0.10` pour 10%
- F-strings pour afficher les variables dans du texte

Exercice 2.3 : Fonction avec Paramètres

Énoncé: Écrivez une fonction qui:

1. Prend deux nombres en paramètre
2. Retourne la plus grande valeur
3. Teste la fonction avec plusieurs exemples

Correction détaillée:

```
# Définir la fonction
def trouver_max(a, b):
    if a > b:
        return a
    else:
        return b

# Ou version plus simple
def trouver_max(a, b):
    if a > b:
        return a
    return b

# Tester la fonction
resultat1 = trouver_max(10, 5)
print("Max entre 10 et 5:", resultat1)

resultat2 = trouver_max(3, 8)
print("Max entre 3 et 8:", resultat2)

resultat3 = trouver_max(7, 7)
print("Max entre 7 et 7:", resultat3)
```

Ce qu'il faut comprendre:

- `def nom_fonction(param1, param2):` définit une fonction
- `return valeur` retourne le résultat
- Appeler la fonction: `resultat = trouver_max(10, 5)`

NIVEAU 3 - AVANCÉ

Exercice 3.1 : Gestion d'Erreurs

Énoncé: Écrivez un programme qui:

1. Demande un nombre à l'utilisateur
2. Demande un diviseur
3. Effectue la division avec gestion d'erreurs
4. Recommence si erreur

Correction détaillée:

```
while True:
    try:
        # Demander les nombres
        nombre = int(input("Entrez un nombre: "))
        diviseur = int(input("Entrez le diviseur: "))

        # Effectuer la division
```

```
    resultat = nombre / diviseur

    print(f"Résultat: {nombre} / {diviseur} = {resultat}")
    break # Sortir de la boucle si succès

except ValueError:
    print("Erreur: Veuillez entrer des nombres valides!")
except ZeroDivisionError:
    print("Erreur: Impossible de diviser par zéro!")
except:
    print("Une erreur inattendue s'est produite!")
```

Ce qu'il faut comprendre:

- `try`: contient le code susceptible de causer une erreur
- `except ValueError`: attrape les erreurs de conversion
- `except ZeroDivisionError`: attrape les divisions par zéro
- `except`: attrape toute autre erreur

Exercice 3.2 : Fonctions Avancées

Énoncé: Écrivez un programme avec plusieurs fonctions:

1. Une fonction qui calcule la factorielle d'un nombre
2. Une fonction qui compte le nombre de voyelles dans un texte
3. Une fonction `main()` qui utilise les deux autres

Correction détaillée:

```
# Fonction 1: Calculer la factorielle
def factorielle(n):
    if n < 0:
        return "Erreur: nombre négatif"
    if n == 0 or n == 1:
        return 1

    resultat = 1
    for i in range(2, n + 1):
        resultat = resultat * i

    return resultat

# Fonction 2: Compter les voyelles
def compter_voyelles(texte):
    voyelles = "aeiouyAEIOUY"
    nombre = 0

    for lettre in texte:
        if lettre in voyelles:
            nombre = nombre + 1
```

```
    return nombre

# Fonction 3: Programme principal
def main():
    print("=== PROGRAMME PRINCIPAL ===")

    # Tester factorielle
    num = 5
    fact = factorielle(num)
    print(f"Factorielle de {num}: {fact}")

    # Tester compteur de voyelles
    phrase = "Bonjour le monde"
    voyelles = compter_voyelles(phrase)
    print(f"Voyelles dans '{phrase}': {voyelles}")

# Exécuter le programme
main()
```

Ce qu'il faut comprendre:

- Décomposer le programme en plusieurs fonctions
- Chaque fonction fait une seule chose
- `return` retourne le résultat à qui a appelé la fonction
- `main()` est le point d'entrée du programme

Exercice 3.3 : Algorithme Complexe

Énoncé: Écrivez un programme qui:

1. Crée une liste de mots
2. Trie les mots par longueur
3. Pour chaque mot, affiche le nombre de voyelles
4. Affiche les statistiques finales

Correction détaillée:

```
def compter_voyelles(mot):
    voyelles = "aeiouyAEIOUY"
    compte = 0
    for lettre in mot:
        if lettre in voyelles:
            compte += 1
    return compte

# Liste de mots
mots = ["python", "programmation", "code", "ordinateur", "informatique"]

# Trier les mots par longueur
```

```
mots_tries = sorted(mots, key=len)

print("=== ANALYSE DES MOTS ===\n")
print("Mots triés par longueur:")
for mot in mots_tries:
    voyelles = compter_voyelles(mot)
    longueur = len(mot)
    print(f" {mot}: {longueur} caractères, {voyelles} voyelles")

# Statistiques
print("\n=== STATISTIQUES ===")
print(f"Nombre total de mots: {len(mots)}")
print(f"Mot le plus court: {mots_tries[0]} ({len(mots_tries[0])} caractères)")
print(f"Mot le plus long: {mots_tries[-1]} ({len(mots_tries[-1])} caractères)")

# Moyenne de voyelles
total_voyelles = 0
for mot in mots:
    total_voyelles += compter_voyelles(mot)
moyenne = total_voyelles / len(mots)
print(f"Moyenne de voyelles par mot: {moyenne:.2f}")
```

Ce qu'il faut comprendre:

- `sorted(liste, key=len)` trie par longueur
- Calculer des statistiques en parcourant les données
- `:.2f` affiche avec 2 décimales

PARTIE 4 : PIÈGES COURANTS D'EXAMEN

PIÈGE 1: L'Indentation

ERREUR COURANTE

```
for i in range(5):
print(i) # Pas d'indentation = ERREUR!
```

BON CODE

```
for i in range(5):
    print(i) # Avec indentation
```

Pourquoi?

Python utilise l'indentation pour savoir quel code appartient au bloc `for`.

PIÈGE 2: Confondre `=` et `==`

✗ ERREUR COURANTE

```
if age = 18: # ERREUR! Utiliser = au lieu de ==
    print("Majeur")
```

✓ BON CODE

```
if age == 18: # Utiliser == pour comparer
    print("Majeur")
```

Pourquoi?

- `=` est pour **assigner** une valeur
- `==` est pour **comparer** deux valeurs

PIÈGE 3: Oublier le Deux-Points

✗ ERREUR COURANTE

```
if age >= 18 # MANQUE le :
    print("Majeur")
```

✓ BON CODE

```
if age >= 18: # Avec le :
    print("Majeur")
```

Où faut-il toujours mettre le deux-points?

- Après `if`, `else`, `elif`
- Après `for`, `while`
- Après `def` (fonction)
- Après `try`, `except`, `finally`

PIÈGE 4: Les Indices Commencent à 0

✗ ERREUR COURANTE

```
liste = ["a", "b", "c"]
print(liste[1]) # Affiche "b" (pas "a"!)
# L'utilisateur pense que l'indice 1 = premier élément
```

✓ BON CODE

```
liste = ["a", "b", "c"]
print(liste[0]) # Affiche "a" (le premier)
print(liste[1]) # Affiche "b" (le deuxième)
```

Souvenez-vous!

Indice	0	1	2
Valeur	"a"	"b"	"c"

PIÈGE 5: len() vs Accès à l'Indice

✗ ERREUR COURANTE

```
liste = [1, 2, 3]
print(liste[len(liste)]) # ERREUR!
# len([1, 2, 3]) = 3, donc liste[3] n'existe pas
# Les indices vont de 0 à 2
```

✓ BON CODE

```
liste = [1, 2, 3]
print(liste[len(liste) - 1]) # Affiche le dernier élément (3)
# ou
print(liste[-1]) # Affiche le dernier élément
```

Pourquoi?

- `len(liste)` retourne le **nombre d'éléments**
- L'indice du **dernier élément** est `len(liste) - 1`

PIÈGE 6: Oublier les Parenthèses de la Fonction

✗ ERREUR COURANTE

```
def dire_bonjour():  
    print("Bonjour")  
  
dire_bonjour # ERREUR! Pas d'appel, pas d'exécution  
  
# On a référencé la fonction, pas l'exécuté
```

✓ BON CODE

```
def dire_bonjour():  
    print("Bonjour")  
  
dire_bonjour() # Avec les parenthèses = exécution
```

PIÈGE 7: Confondre print() et return

✗ ERREUR COURANTE

```
def calculer(a, b):  
    print(a + b) # AFFICHE seulement, ne retourne rien  
  
resultat = calculer(5, 3) # resultat = None!  
print(resultat) # Affiche None
```

✓ BON CODE

```
def calculer(a, b):  
    return a + b # RETOURNE le résultat  
  
resultat = calculer(5, 3) # resultat = 8  
print(resultat) # Affiche 8
```

Différence clé!

	print()	return
Affiche à l'écran	✓ Oui	✗ Non
Permet de récupérer la valeur	✗ Non	✓ Oui
Retourne une valeur	✗ Non	✓ Oui

PIÈGE 8: Les Espaces et l'Indentation

✗ ERREUR COURANTE

```
# Mélanger des espaces et des tabulations
def fonction():
    print("1")
    print("2") # Tabulation au lieu d'espaces = ERREUR!
```

✓ BON CODE

```
def fonction():
    print("1")
    print("2") # Toujours des espaces (ou toujours des tabulations)
```

Conseil d'examen

À la main sur papier, dessinez les **4 espaces** clairement!

PIÈGE 9: Modifier une Liste Pendant qu'on la Parcourt

✗ ERREUR COURANTE

```
nombres = [1, 2, 3, 4, 5]

for nombre in nombres:
    if nombre == 3:
        del nombres[nombres.index(nombre)] # Très mauvais!
        # Cela modifie la liste pendant la boucle
```

✓ BON CODE

```
nombres = [1, 2, 3, 4, 5]
nouvelles = []

for nombre in nombres:
    if nombre != 3:
        nouvelles.append(nombre)

nombres = nouvelles
# ou créer une nouvelle liste sans le 3
```

PIÈGE 10: Oublier les Guillemets pour les Textes

✗ ERREUR COURANTE

```
nom = Alice # ERREUR! Alice n'est pas défini

# Python pense que Alice est une variable
```

✓ BON CODE

```
nom = "Alice" # Avec les guillemets
print(nom) # Affiche: Alice
```

PIÈGE 11: Diviser par Zéro

✗ ERREUR COURANTE

```
resultat = 10 / 0 # ERREUR: ZeroDivisionError
```

✓ BON CODE

```
try:
    resultat = 10 / 0
except ZeroDivisionError:
    print("Impossible de diviser par zéro!")
```

PIÈGE 12: Accéder à une Clé Inexistante dans un Dictionnaire

✗ ERREUR COURANTE

```
personne = {"nom": "Alice"}
print(personne["age"]) # ERREUR: KeyError
```

✓ BON CODE

```
personne = {"nom": "Alice"}

# Méthode 1: Vérifier d'abord
```

```
if "age" in personne:
    print(personne["age"])

# Méthode 2: Utiliser get()
age = personne.get("age", "Non trouvé")
print(age) # Affiche: Non trouvé

# Méthode 3: Utiliser try/except
try:
    print(personne["age"])
except KeyError:
    print("Clé inexistante!")
```

PIÈGE 13: Changer le Type de Donnée par Erreur

✗ ERREUR COURANTE

```
age = "25" # C'est du texte
resultat = age + 5 # ERREUR: "25" + 5
# TypeError: cannot add str and int
```

✓ BON CODE

```
age = int("25") # Convertir en nombre
resultat = age + 5 # 25 + 5 = 30
print(resultat) # Affiche: 30
```

PIÈGE 14: Les Conditions avec AND/OR

✗ ERREUR COURANTE

```
age = 25

# Mauvaise logique
if age > 18 and age < 65:
    print("Âge de travail")
# Cela marche mais n'est pas optimal

# Très mauvais
if age > 18 and age > 100:
    print("?") # Impossible: si > 18 ET > 100
```

✓ BON CODE

```
age = 25

if age >= 18 and age < 65:
    print("Âge de travail")

# Ou
if 18 <= age < 65:
    print("Âge de travail")
```

PIÈGE 15: Oublier le return dans une Fonction

✗ ERREUR COURANTE

```
def doubler(nombre):
    resultat = nombre * 2
    # Pas de return!

valeur = doubler(5)
print(valeur) # Affiche: None
```

✓ BON CODE

```
def doubler(nombre):
    resultat = nombre * 2
    return resultat # N'oubliez pas!

valeur = doubler(5)
print(valeur) # Affiche: 10
```



RÉSUMÉ FINAL POUR L'EXAMEN

À absolument écrire correctement à la main:

1. **L'indentation** - 4 espaces minimum
2. **Les deux-points** après if, for, while, def, try
3. **Les ==** pour comparer (pas =)
4. **Les guillemets** autour des textes
5. **Les parenthèses** pour appeler une fonction
6. **Le return** dans les fonctions si vous voulez retourner une valeur

À toujours vérifier:

- Les indices commencent à **0**
- `len()` retourne le **nombre** d'éléments (pas l'indice du dernier)
- `print()` affiche, `return` retourne
- Les erreurs courant: `ValueError`, `ZeroDivisionError`, `IndexError`, `KeyError`
- Traiter les erreurs avec `try/except`

Les 3 structures principales:

```
# 1. CONDITION
if condition:
    # code
elif condition2:
    # code
else:
    # code

# 2. BOUCLES
for i in range(10):
    # code

while condition:
    # code

# 3. FONCTION
def nom_fonction(param):
    # code
    return resultat
```



CONSEILS POUR L'EXAMEN ÉCRIT:

1. **Écrivez lisiblement** - Le correcteur doit pouvoir lire votre indentation
2. **Testez mentalement votre code** - Parcourez-le pas à pas
3. **Commentez si vous avez des doutes** - Un commentaire clair aide le correcteur
4. **Utilisez des noms de variables explicites** - `age` et pas `a`, `prix_total` et pas `pt`
5. **Structurez votre code** - Allez du simple au compliqué
6. **Vérifiez les cas limites** - Que se passe-t-il avec 0? Avec une liste vide?
7. **Si vous faites une erreur, barrez** - Ne gomez pas votre copie d'examen



BONNE CHANCE À L'EXAMEN!

Vous avez maintenant tous les outils pour réussir. Concentrez-vous sur la clarté et la précision!